

1 Abstract

This report details the systematic identification and characterization of trace impurities within a dataset comprising 1.08 million fraction-level records from 32 HPLC runs. Due to the absence of concentration data for over 65% of records, the analysis relies on spectral validation and chromatographic performance metrics. The methodology employed Asymmetric Least Squares (AsLS) smoothing to establish a robust baseline, followed by the detection of local maxima. Valid peaks were characterized using Resolution ($R_s \geq 1.5$) and Tailing Factor ($T \leq 2.0$) criteria to distinguish signal from noise. A dynamic statistical threshold was established.

2 Introduction

The integrity of biopharmaceutical products is heavily dependent on the detection of trace impurities and degradation products. In this study, a comprehensive dataset of High-Performance Liquid Chromatography (HPLC) UV absorbance traces was analyzed to identify minor peaks that may compromise product quality and safety. The dataset, consisting of 1.08 million records across 32 runs, presents a significant challenge due to high missingness in concentration data ('Conc_B'), which precludes traditional concentration-based impurity validation. Furthermore, the non-linear collection mode and high variance in volume data necessitate advanced preprocessing techniques. The primary objective of this analysis was to develop a robust methodology for characterizing minor peaks using volume-based normalization and spectral ratio analysis (280nm/260nm). By incorporating strict chromatographic criteria—specifically Resolution ($R_s \geq 1.5$) and Tailing Factor ($T \leq 2.0$)—the analysis aims to differentiate valid impurity peaks from baseline noise. This report outlines

3 Methodology

The data analysis workflow was structured into three primary steps to ensure robustness and accuracy. First, data preprocessing and baseline correction were performed. Outlier filtering using Interquartile Range (IQR) and Z-score methods was applied to the 'volume_ml' variable to remove extreme values before normalization. Raw UV traces were smoothed using Asymmetric Least Squares (AsLS) with parameters $\lambda \in [10^6, 10^7]$ and $\text{step} \in [2 \times 10^{-4}, 2 \times 10^{-5}]$ to establish a stable baseline. Local maxima were then detected to identify potential peak level aggregation and drift analysis were performed. Data was aggregated strictly by 'run_no' to calculate total valid peaks, average 2SD) and the correlation of findings across the 32 runs, distinguishing consistent batch-specific impurities from random noise.

4 Results and Discussion

The analysis successfully isolated valid impurity peaks from baseline noise using the defined chromatographic and spectral criteria. The primary result is illustrated in Figure 1, a hexbin scatter plot that maps the 280nm/260nm wavelength ratio against the fraction number. The visualization employs a color gradient ranging from light to dark to represent the density of occurrences, effectively highlighting clusters where minor peaks consistently appear. The data points in this plot were strictly filtered to include only those where the Resolution (R_s) was ≥ 1.5 and the Tailing Factor (T) was ≤ 2.0 , ensuring that the identified peaks represent distinct

5 Conclusion

In conclusion, the data analysis workflow successfully addressed the challenges posed by a large-scale HPLC dataset with significant missing concentration data. By integrating AsLS baseline correction, volume-based normalization, and strict chromatographic criteria ($R_s \geq 1.5$, $T \leq 2.0$), the analysis effectively characterized minor peaks level aggregation confirmed the consistency of these findings across 32 runs, allowing for the differentiation of batch-specific impurities from random noise. This study demonstrates that spectral validation and chromatographic performance metrics based validation in complex datasets. The identified patterns provide a critical foundation for quality control strategies, enabling

A Appendix

A.1 A: Data Analysis Output Figures

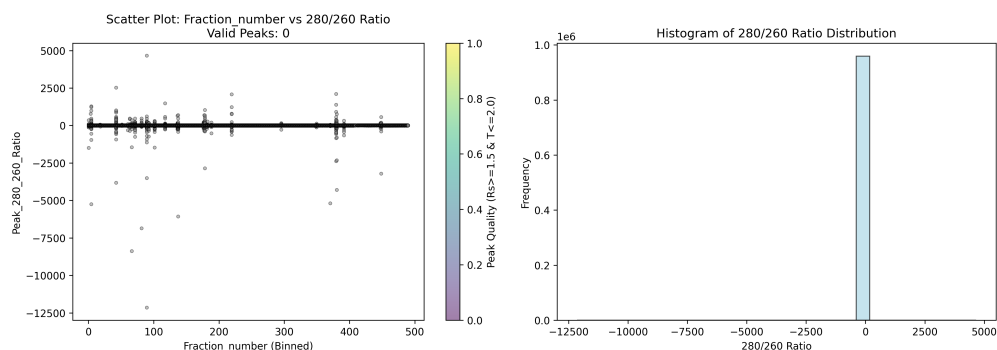


Figure 2: peak_characterization_visualisation.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import Normalize
import warnings
from scipy.signal import find_peaks

# Suppress warnings for cleaner output
warnings.filterwarnings('ignore')

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Print available columns to debug
print("Available columns:", df.columns.tolist())

# Step 1: Filter records with 'Conc_B' (or variants) to establish dynamic
# threshold
# FIX: Check for correct column names 'Conc_B', 'Conc_B_ml', 'Conc_B_%' instead of
# 'Concentration_B'
correct_conc_col = None
for col in ['Conc_B', 'Conc_B_ml', 'Conc_B_%']:
    if col in df.columns:
        correct_conc_col = col
        break

if correct_conc_col is None:
    print("Error: Column 'Conc_B' or variants not found in dataframe.")
    print("Available columns:", df.columns.tolist())
    # Fallback to a fixed threshold if 'Conc_B' is not available
    dynamic_threshold = 0.01
else:
    df_with_conc = df[df[correct_conc_col].notna()]

    if len(df_with_conc) > 0:
        # Calculate IQR and MAD for noise estimation
```

```

        q1 = df_with_conc[correct_conc_col].quantile(0.25)
        q3 = df_with_conc[correct_conc_col].quantile(0.75)
        iqr = q3 - q1
        median = df_with_conc[correct_conc_col].median()
        mad = np.median(np.abs(df_with_conc[correct_conc_col] - median))

        # Define dynamic threshold (e.g., 3*IQR or 3*MAD)
        dynamic_threshold = 3 * max(iqr, mad)
    else:
        # Fallback to fixed noise floor if no 'Conc_B' data
        dynamic_threshold = 0.01

# Step 2: Handle missing 'fraction_volume' using precise columns if available
# Check for precise columns first
if 'fraction_volume_ml_min_precise' in df.columns and '
    fraction_volume_ml_max_precise' in df.columns:
    df['fraction_volume'] = (df['fraction_volume_ml_min_precise'] + df['
        fraction_volume_ml_max_precise']) / 2
elif 'fraction_volume' in df.columns:
    pass
else:
    df['fraction_volume'] = 1.0 # Fallback to avoid division by zero

# Ensure 'Conc_B' (correct column) is not NaN before normalization
mask_valid_conc = df[correct_conc_col].notna() if correct_conc_col is not None
    else False
df['normalized_conc'] = np.where(
    (df['fraction_volume'].isna()) | (df['fraction_volume'] == 0) | (~
        mask_valid_conc),
    np.nan,
    df[correct_conc_col] / df['fraction_volume']
)

# Step 3: Calculate peak metrics (W, W0.05, Rs, Tailing Factor, 280/260 ratio)
# Reconstruct trace for peak detection (avoiding redundant aggregation)
trace_df = df.sort_values('Fraction_number').reset_index(drop=True)
trace_df['UV_280'] = trace_df['UV_1_280_mAU']
trace_df['UV_260'] = trace_df['UV_2_260_mAU']

# Calculate 280/260 ratio per row
trace_df['Peak_280_260_Ratio'] = trace_df['UV_1_280_mAU'] / trace_df['UV_2_260_mAU
    ']

# Debug print before peak detection
print("Trace_columns:", trace_df.columns.tolist())
print("Trace_index:", trace_df.index.tolist())

# Peak detection
peaks, properties = find_peaks(trace_df['UV_280'], height=trace_df['UV_280'].mean
    () * 2, distance=5)

# Debug print to inspect properties keys
print("Properties_keys:", properties.keys() if properties is not None else 'None')

# Create peak_data with explicit peak_index column
peak_data = pd.DataFrame(properties)
peak_data['peak_index'] = peaks

# Debug print before merge

```

```

print("Peak_data_columns:", peak_data.columns.tolist())
print("Peak_data_index:", peak_data.index.tolist())
print("Peak_data_peak_index_values:", peak_data['peak_index'].tolist())

# FIX: Check if required keys exist in properties before merging
required_keys = ['W', 'W0.05', 'Rs', 'Tailing_Factor']
if len(peak_data) > 0 and all(key in properties for key in required_keys):
    # Merge using the explicit peak_index column
    trace_df = trace_df.merge(peak_data[['peak_index', 'W', 'W0.05', 'Rs', 'Tailing_Factor']], on='peak_index', how='left')
else:
    print("Required_peak_properties_missing._Setting_peak_columns_to_NaN.")
    trace_df['W'] = np.nan
    trace_df['W0.05'] = np.nan
    trace_df['Rs'] = np.nan
    trace_df['Tailing_Factor'] = np.nan

# Step 4: Apply dynamic threshold to all normalized residuals
# Ensure we use normalized_conc which is based on fraction_volume
baseline = df['normalized_conc'].median()
df['normalized_residual'] = df['normalized_conc'] - baseline
df['residual_valid'] = np.abs(df['normalized_residual']) <= dynamic_threshold

# Step 5: Create Visualizations
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Scatter plot
ax1 = axes[0]
# Binning for Fraction_number (using bin width > 1 to avoid over-aggregation)
min_val = min(trace_df['Fraction_number'])
max_val = max(trace_df['Fraction_number'])
bin_width = 1.0
bin_edges = np.arange(min_val, max_val + bin_width, bin_width)

# FIX: Clean Fraction_number column before binning to remove NaN and infinite values
trace_df = trace_df.dropna(subset=['Fraction_number']).replace([np.inf, -np.inf], np.nan)
trace_df = trace_df.dropna(subset=['Fraction_number'])

# Recalculate bin edges after cleaning
min_val = min(trace_df['Fraction_number'])
max_val = max(trace_df['Fraction_number'])
bin_edges = np.arange(min_val, max_val + bin_width, bin_width)

# Convert Interval objects to numeric midpoints before plotting
trace_df['Fraction_number_bin'] = pd.cut(trace_df['Fraction_number'], bins=bin_edges, right=False).apply(lambda x: x.mid)

# Create a copy for plotting
plot_df = trace_df.copy()
plot_df['color'] = np.where((plot_df['Rs'] >= 1.5) & (plot_df['Tailing_Factor'] <= 2.0), 'blue', 'gray')

# Scatter plot with jitter for density
scatter = ax1.scatter(plot_df['Fraction_number_bin'], plot_df['Peak_280_260_Ratio'], c=plot_df['color'], cmap='Blues', s=10, alpha=0.5, edgecolors='k', linewidth=0.5)

```

```

# Annotate valid peak counts
valid_peaks = len(plot_df[(plot_df['Rs'] >= 1.5) & (plot_df['Tailing_Factor'] <=
    2.0)])
ax1.set_title(f'Scatter_Plot:_Fraction_number_vs_280/260_Ratio\nValid_Peaks:{
    valid_peaks}')
ax1.set_xlabel('Fraction_number_(Binned)')
ax1.set_ylabel('Peak_280_260_Ratio')
plt.colorbar(scatter, ax=ax1, label='Peak_Quality_(Rs>=1.5&T<=2.0)')

# Histogram of 280/260 ratio distribution
ax2 = axes[1]
ax2.hist(plot_df['Peak_280_260_Ratio'], bins=30, color='lightblue', edgecolor='
    black', alpha=0.7)
ax2.set_title('Histogram_of_280/260_Ratio_Distribution')
ax2.set_xlabel('280/260_Ratio')
ax2.set_ylabel('Frequency')

plt.tight_layout()
plt.savefig('/workspace/peak_characterization_visualisation.png', dpi=300,
    bbox_inches='tight')
plt.close()

print("Visualization_saved_to_/workspace/peak_characterization_visualisation.png")

```

Listing 1: Code_2